



Staix API 사용 가이드

목차

1. 기본 가이드

1.1. OpenAI SDK 최신화

1.2. Base URL 설정

1.3. API Key 확인

1.4. 모델 선택 및 코드 스니펫 ...

1.4.1 Chat Completion 엔드...

1.4.2 Response 엔드포인트 ...

1.4.3 Embedding 엔드포인...

1.4.4 gemini-2.5-flash-im...

2. API 상세 명세

2.1. 엔드포인트 요약

2.2. 지원 파라미터

2.3. Request, Response F...

2.4. 모델 기능

3. 오류 처리 가이드

4. LangChain

4.1. init_chat_model 사용 ...

4.2. ChatOpenAI 사용 예시

5. Codex CLI(Experimental)

5.1. 빠른 시작

- AI 게이트웨이는 LLM을 포함한 다양한 AI 모델을 **하나의 표준 인터페이스로 통합하여 제공합니다.**
- 이를 통해 사용자는 **OpenAI 형식의 입력/출력을 그대로 사용**하면서도, 여러 Provider의 모델을 간편하게 호출할 수 있습니다.

1. 기본 가이드

본 섹션은 렛서의 AI 게이트웨이를 처음 사용하는 사용자를 위한 기본 연동 가이드입니다.

아래 단계에 따라 SDK 설치부터 API 호출까지 순서대로 진행할 수 있습니다.

1.1. OpenAI SDK 최신화

AI 게이트웨이는 OpenAI SDK와 호환됩니다.

따라서 OpenAI SDK를 설치하거나 최신 버전으로 업데이트해야 합니다.

```
pip install --upgrade openai
```

⚠ 주의: 반드시 최신 버전을 사용해야 모든 기능이 정상 동작합니다.

1.2 Base URL 설정

[5.2. 주요 파일 및 설명](#)[5.3. 배치 위치](#)[5.4. 참고](#)[6. gemini-2.5-flash-image/ge...](#)[6.1. 주요 기능 요약](#)[6.2. Request / Response ...](#)[✓ Request 예시](#)[✓ Response 예시](#)[6.3. Python 예제 코드](#)[6.4 TypeScript 예제 코드](#)

1.2. Basic URL

AI 게이트웨이를 사용하기 위해서는 `base_url` 을 지정해야 합니다.

예시:

```
client = openai.OpenAI( base_url="https://gateway.letsur.ai/v1", # Letsur Gateway API URL api_key="YOUR_API_KEY" #  
발급받은 API Key )
```

`base_url` 은 반드시 "https://gateway.letsur.ai/v1" 로 지정합니다.

"지원 모델 - 모델 별 코드보기" 에서 자세한 사용 예시를 보실 수 있습니다.

1.3. API Key 확인

- "API 키" 메뉴에서 자신의 API Key를 확인할 수 있습니다.
- 발급받은 Key는 반드시 안전하게 보관하세요. (환경 변수 사용 권장)

예시 (환경 변수 등록):

```
export LETSUR_API_KEY="발급받은_키"
```

파이썬 코드에서 사용:

```
import os client = openai.OpenAI( base_url="https://gateway.letsur.ai/v1", api_key=os.environ["LETSUR_API_KEY"] )
```

1.4. 모델 선택 및 코드 스니펫 실행

AI 게이트웨이는 다양한 모델을 지원하며, 모델 리스트와 엔드포인트는 "지원 모델" 메뉴에서 확인할 수 있습니다.

가장 간단한 예시는 `chat.completions.create` API를 활용하는 것입니다.

```
# 사용할 모델 지정 model = "gpt-4o" # 모델 리스트에서 선택 # Chat Completion 호출 response = client.chat.completions.create(
model=model, messages=[ {"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content":
"Hello! How are you?"} ], ) # 응답 출력 print(response.choices[0].message.content)
```

실행 결과:

I'm doing well, thank you! How can I help you today?

⚠ 주의: Response 엔드포인트는 프리뷰 버전이며, 일부 기능에 오류가 있을 수 있습니다.

- ▶ 1.4.1 Chat Completion 엔드포인트 코드 예시
- ▶ 1.4.2 Response 엔드포인트 (Preview) 코드 예시
- ▶ 1.4.3 Embedding 엔드포인트 코드 예시
- ▶ 1.4.4 gemini-2.5-flash-image/gemini-3-pro-image-preview 예시 코드

2. API 상세 명세

2.1. 엔드포인트 요약

Note: Responses API는 프리뷰라 간헐적 비호환/에러가 있을 수 있습니다. 안정성 우선이면 /chat/completions 사용을 권장합니다.

 Response Endpoint는 OpenAI의 모델만 지원합니다.

--	--	--

기능	메소드	경로
Chat Completions	POST	v1/chat/completions
Embeddings	POST	v1/embeddings
Responses (Preview)	POST	v1/responses

2.2. 지원 파라미터

OpenAI의 모든 공식 파라미터를 기본적으로 지원합니다.
단, 일부 불안정하거나 비호환되는 파라미터는 내부적으로 필터링됩니다.

- /v1/chat/completions
- * 상세 내용은 [OpenAI Chat Completions Docs](#) 참고

파라미터	설명
messages	대화 메시지 배열
model	사용할 모델명
temperature, top_p, n	샘플링 관련 설정
max_tokens, max_completion_tokens	출력 토큰 제한
stream, stream_options	스트리밍 응답 옵션
presence_penalty, frequency_penalty	토큰 반복 제어
logprobs, top_logprobs	확률 로그 반환
tools, tool_choice, functions	함수 호출/도구 호출 제어
response_format	응답 형식 (json/text 등)
reasoning_effort	추론 강도 (reasoning 모델용)
parallel_tool_calls	병렬 도구 호출 설정
web_search_options	웹 검색 통합 시 옵션

- **/v1/embeddings**

파라미터	설명
input	임베딩할 텍스트
model	임베딩 모델명
encoding_format	반환 형식 (float / base64)
user	사용자 식별자
dimensions	임베딩 차원 (지원 모델 한정)

- **/v1/responses (Preview)**

* 상세 내용은 [OpenAI Responses API Docs](#) 참고

파라미터	설명
prompt / input / text	모델 입력
model	사용할 모델
temperature, top_p	생성 제어 파라미터
max_output_tokens	최대 출력 토큰 수
reasoning	Reasoning 옵션
parallel_tool_calls	병렬 도구 호출 설정
previous_response_id	이전 응답 연동 시 사용
instructions	시스템 지침
include	응답에 포함할 추가 정보

2.3. Request, Response Format

- `/v1/chat/completions`

- Requests

```
import openai # OpenAI 클라이언트 초기화 client = openai.OpenAI( base_url="https://gateway.letsur.ai/v1", # Letsur의 API URL로 변경 api_key="YOUR_API_KEY" # Letsur의 API Key로 변경 ) # 사용할 모델 지정 model = "claude-3-5-haiku-20241022" # chat.completions.create API 호출 response = client.chat.completions.create( model=model, messages=[ {"role": "system", "content": "You are a bot"}, {"role": "user", "content": "How are you?"} ], ) # 응답 출력 print(response.choices[0].message.content)
```

- Responses

```
{ "id": "chatcmpl-xxxx", "object": "chat.completion", "created": 1736567890, "model": "gpt-4o", "choices": [ { "index": 0, "message": {"role": "assistant", "content": "..."}, "finish_reason": "stop" } ], "usage": { "prompt_tokens": 12, "completion_tokens": 42, "total_tokens": 54 } }
```

- `/v1/embeddings`

- Requests

```
import openai # OpenAI 클라이언트 초기화 client = openai.OpenAI( base_url="https://gateway.letsur.ai/v1", # Letsur의 API URL로 변경 api_key="YOUR_API_KEY" # Letsur의 API Key로 변경 ) # 사용할 임베딩 모델 지정 model = "text-embedding-3-large" # embeddings.create API 호출 response = client.embeddings.create( input="Your text string goes here", model=model ) # 생성된 임베딩 벡터 출력 print(response.data[0].embedding)
```

- Responses

SDK는 아래와 유사한 객체를 반환합니다. JSON이 필요하다면 `resp.model_dump_json()` 을 사용하세요.

```
CreateEmbeddingResponse( data=[Embedding(embedding=[0.0037610608401334015, -0.010516710450177726,
```



```
createEmbeddingResponse( data=[Embedding(embedding=[0.00376107, -0.01051621, -0.00310958, 0.04576122, ...], index=0, object='embedding')], model='text-embedding-3-large', object='list',
usage=Usage(prompt_tokens=5, total_tokens=5, completion_tokens=0, completion_tokens_details=None,
prompt_tokens_details=None))
```

```
{ "object": "list", "data": [ { "object": "embedding", "index": 0, "embedding": [0.00376107, -0.01051621, -0.00310958, 0.04576122, ...] } ], "model": "text-embedding-3-large", "usage": { "prompt_tokens": 5, "total_tokens": 5, "completion_tokens": 0 } }
```

- **/v1/responses (Preview)**

- Requests

```
import openai # OpenAI 클라이언트 초기화 client = openai.OpenAI( base_url="https://gateway.letsur.ai/v1", #
Letsur의 API URL로 변경 api_key="YOUR_API_KEY" # Letsur의 API Key로 변경 ) # 사용할 모델 지정 model = "o3-mini" #
모델에 전달할 지침(시스템 프롬프트 역할) instruction = ( "You are a personal math tutor. " "When asked a math
question, run code to answer the question." "After solving questions, re-think about given answer and run
another code that can confirm given answer." ) # 모델에 전달할 사용자 입력 input_text = "I need to solve the
equation `3x + 11 = 14`. Can you help me?" # responses.create API 호출 response = client.responses.create(
model=model, instructions=instruction, input=input_text ) # 응답 출력
print(response.output[-1].content[0].text)
```

- Responses

```
Response( id='resp_xxx', created_at=1760336603.0, error=None, incomplete_details=None, instructions=(
"You are a personal math tutor. " "When asked a math question, run code to answer the question. " "After
solving questions, re-think about given answer and run another code " "that can confirm given answer." ),
metadata={}, model='o3-mini-2025-01-31', object='response', output=[ ResponseReasoningItem( id='rs_xxx',
summary=[], type='reasoning', status=None, content=None, encrypted_content=None ), ResponseOutputMessage(
id='msg_xxx', role='assistant', type='message', status='completed', content=[ ResponseOutputText(
type='output_text', annotations=[], logprobs=[], text=( "Let's solve the equation step by step:\n\n" "1.
We start with the equation:\n" " 3x + 11 = 14\n\n" "2. Subtract 11 from both sides:\n" " 3x = 14 - 11\n"
" 3x = 3\n\n" "3. Divide both sides by 3:\n" " x = 3 / 3\n" " x = 1\n\n" "So, the solution to the
equation is x = 1.\n\n" "To confirm this result, I'll run a Python code snippet using Sympy:\n" "-----
```

```
-----\n" "[Run Code]\n" "-----\n\n-----\n" "import sympy as sp\n" "x = sp.symbols('x')\n" "equation = sp.Eq(3*x + 11, 14)\n" "solution = sp.solve(equation, x)\n" "solution\n" "-----\n\n\n" "The code confirms that the solution is x = 1.\n\n" "Thus, the answer is x = 1." ) ) ] ) ], parallel_tool_calls=True, temperature=1.0, tool_choice='auto', tools=[], top_p=1.0, max_output_tokens=None, previous_response_id=None, reasoning=Reasoning( effort='medium', generate_summary=None, summary=None ), service_tier='default', status='completed', text=ResponseTextConfig( format=ResponseFormatText(type='text'), verbosity='medium' ), truncation='disabled', usage=ResponseUsage( input_tokens=70, input_tokens_details=InputTokensDetails( cached_tokens=0, audio_tokens=None, text_tokens=None ), output_tokens=329, output_tokens_details=OutputTokensDetails( reasoning_tokens=128, text_tokens=None ), total_tokens=399 ), user=None, store=True, background=False, billing={'payer': 'developer'}, max_tool_calls=None, prompt_cache_key=None, safety_identifier=None, top_logprobs=0 )
```

2.4. 모델 기능

Image / Reasoning 기능을 지원하는 모델의 경우, 이미지와 텍스트를 함께 전송하여 분석을 요청할 수 있습니다. 각 기능의 Request code 예시는 “지원 모델” 메뉴에서 각 모델들의 “코드 보기”를 통해 확인하실 수 있습니다.

- Image 기능
 - 모델 중, image 입력이 가능한 모델 분류
 - ▶ Image 입력 Request 예시
- Reasoning 기능
 - 모델 중, 추론을 할 수 있는 Reasoning 모델 분류
 - ✱ [Reasoning Responses](#)

3. 오류 처리 가이드

API 요청 시 발생할 수 있는 주요 HTTP 상태 코드 및 에러 메시지와 그 의미는 다음과 같습니다.

Status Code	Error Type	Description
400	BadRequestError	지원되지 않거나 잘못된 요청 형식이 전달될 때 발생합니다.

400	UnsupportedParamsError	지원되지 않는 파라미터가 요청에 포함되었을 때 발생합니다.
400	ContextWindowExceededError	입력 메시지의 길이가 모델의 최대 컨텍스트 윈도우를 초과했을 때 발생하는 오류입니다. (자동 축약이나 fallback 처리를 위해 사용됩니다.)
400	ContentPolicyViolationError	콘텐츠 정책(예: 민감한 내용, 금지된 요청 등)에 위배되는 메시지가 포함되었을 때 발생하는 오류입니다
400	ImageFetchError	이미지 데이터를 불러오거나 처리하는 과정에서 오류가 발생했을 때 발생합니다.
400	InvalidRequestError	더 이상 사용되지 않는 오류 유형으로, 대신 BadRequestError 가 사용됩니다. (Deprecated)
401	AuthenticationError	인증에 실패했을 때 발생합니다. (API 키가 없거나 잘못된 경우)
403	PermissionDeniedError	권한이 부족하거나 접근이 금지된 리소스에 접근하려 할 때 발생합니다.
404	NotFoundError	존재하지 않는 모델이나 경로를 요청했을 때 발생합니다. (예: 잘못된 모델 이름 gpt-8)
408	Timeout	요청 처리 중 제한된 시간이 초과되었을 때 발생합니다.
422	UnprocessableEntityError	요청은 유효하지만 서버가 해당 데이터를 처리할 수 없을 때 발생합니다. (입력 값이 처리 불가능한 경우 등)
429	RateLimitError	호출 빈도나 쿼터 한도를 초과했을 때 발생합니다.
500	APIConnectionError	서버 연결 오류가 발생했을 때 반환됩니다.
500	APIError	일반적인 500번대 내부 서버 오류입니다.
500	InternalServerError	정의되지 않은 오류가 발생할 때 반환됩니다.
501	NotImplemented	endpoint 는 지원하지만 기능은 아직 지원하지 않을 경우 발생하는 오류입니다
503	ServiceUnavailableError	서버(또는 LLM 공급자)가 일시적으로 서비스를 제공할 수 없을 때 발생합니다.

4. LangChain

base_url 과 api_key 를 지정하면 Gateway를 통해 원하는 모델을 불러올 수 있습니다.

 주의: AI Gateway는 다양한 Provider의 모델을 OpenAI API 형식으로 표준화하여 제공 합니다. 따라서 provider 를 openai 로 지정해야 합니다.

4.1. init_chat_model 사용 예시

```
from langchain.chat_models import init_chat_model llm = init_chat_model( model="gemini-2.5-flash", model_provider="openai", # provider를 openai로 지정 temperature=0, max_tokens=4096, base_url="https://gateway.letsur.ai/v1", api_key=API_KEY ) response = llm.invoke("Hello, world!") print(response)
```

4.2. ChatOpenAI 사용 예시

```
from langchain_openai import ChatOpenAI llm = ChatOpenAI( model="gemini-2.5-flash", temperature=0, max_tokens=None, timeout=None, base_url="https://gateway.letsur.ai/v1", api_key=API_KEY ) response = llm.invoke("Hello, world!") print(response)
```

5. Codex CLI(Experimental)

본 섹션은 Codex CLI를 AI 게이트웨이와 함께 사용하기 위한 최소 설정 방법을 제공합니다. 아래 단계에 따라 환경을 구성하면 AI 게이트웨이를 통해 Codex CLI를 사용할 수 있습니다.

5.1. 빠른 시작

- 1. 공식 문서의 가이드를 따라 Codex CLI를 설치합니다.
- 2. 아래의 "주요 파일 및 설명"을 확인합니다.
- 3. \$HOME/.codex 폴더에 config.toml 과 .env 파일을 설정합니다.
- 4. 아래 명령어를 실행하여 Codex를 실행합니다.

```
codex
```

5.2. 주요 파일 및 설명

-  `.codex/config.toml`

Codex CLI가 어떤 모델을 사용할지를 정의하는 설정 파일입니다.

AI 게이트웨이와 연결하기 위한 예시는 다음과 같습니다:

- [기본 Config]

```
model = "gpt-5" model_provider = "letsur-ai-gateway" [model_providers.letsur-ai-gateway] # Codex UI에 표시
될 제공자 이름 name = "OpenAI using Chat Completions" # 채팅 완성 요청(POST)을 위해 이 URL 뒤에 `/chat/completions`
경로가 붙습니다. base_url = "https://gateway.letsur.ai/v1" # `env_key`가 설정된 경우, 이 제공자를 사용할 때 반드시 설정해
야 하는 환경 변수의 이름입니다. # 환경 변수 값이 비어 있지 않아야 하며, POST 요청의 HTTP 헤더 Authorization에 # `Bearer TOKE
N` 형태로 사용됩니다. env_key = "LETSUR_AI_GATEWAY_KEY" # wire_api는 "chat" 또는 "responses" 값을 가질 수 있으며, 생
략 시 기본값은 "chat"입니다. wire_api = "chat" # 필요한 경우 URL에 추가할 쿼리 파라미터를 지정합니다. query_params = {}
```

⚠ 주의: AI 게이트웨이를 통해 GPT-5 모델 이외에도 다양한 LLM 들을 연동하여 이용하실 수 있습니다. 일부 모델은 제대로 연동되지 않
을 수 있습니다.

- 위와 같이 설정하면 `codex` 실행 시 기본 모델(`gpt-5`)이 자동으로 사용됩니다.
- 실행 중에는 `/models` 또는 CLI 옵션을 통해 기존 Codex 기본 Preset으로 전환할 수 있습니다.

- [커스텀 모델 프리셋 설정 (기본 Config 확장)]

여러 모델을 프리셋(profile) 형태로 등록할 수 있으며, 실행 시 `-p` 옵션으로 지정 가능합니다.

```
# Claude 4 Sonnet [profiles.claude-4-sonnet] model = "claude-sonnet-4-20250514" model_provider = "letsur-
ai-gateway" # Claude 4 Opus [profiles.claude-4-opus] model = "claude-opus-4-20250805" model_provider = "l
etsur-ai-gateway" model_reasoning_effort = "high" # Gemini 2.5 Pro [profiles.gemini-2-5-pro] model = "gem
ini-2.5-pro" model_provider = "letsur-ai-gateway" model_reasoning_effort = "medium"
```

실행 예시:

```
codex -p claude-4-opus
```

⚠ 일부 모델 또는 파라미터는 게이트웨이 지원 여부에 따라 동작하지 않을 수 있습니다.

-  `.codex/.env`
 - 실행에 필요한 환경 변수를 정의하는 파일입니다. 위 `config.toml` 에서 `env_key` 로 지정한 변수(`LETSUR_AI_GATEWAY_KEY`)를 여기서 설정해야 합니다.

예시:

```
LETSUR_AI_GATEWAY_KEY=<letsur-gateway-api-key>
```

5.3. 배치 위치

- `$HOME/.codex/config.toml` 파일을 두면 Codex CLI가 자동으로 인식합니다.
- 이 폴더 내의 `.codex` 를 복사한 뒤, 환경에 맞게 값만 수정하면 바로 사용 가능합니다.

5.4. 참고

- Codex CLI의 실제 사용 방법 및 옵션은 버전에 따라 달라질 수 있습니다.
- 항상 [공식 문서](#)와 최신 버전을 확인하시기 바랍니다.

6. qemini-2.5-flash-image/qemini-3-pro-image-preview (나노바나나)

- ! gemini-3-pro-image-preview 모델의 경우, 제공사 (Google) 측 불안정성이 존재하는 점을 이용에 참고 부탁드립니다.

기능	설명
텍스트 → 이미지 생성 (Text-to-Image)	텍스트 프롬프트로부터 이미지를 빠르게 생성
이미지 → 이미지 변환 (Image-to-Image)	입력 이미지를 기반으로 스타일 변경 및 편집 가능
초저지연(Flash) 응답	기존 모델 대비 매우 빠른 응답 속도
완전 호환 구조	<code>/v1/chat/completions</code> 과 완전히 동일한 구조로 동작
자동 모델 라우팅	<code>model="gemini-2.5-flash-image"</code> 혹은 <code>model="gemini-3-pro-image-preview"</code> 지정 시 자동 라우팅

응답 형식: 이미지가 포함될 경우 **images** 필드(base64) 추가

- Input = Text

```
import openai
client = openai.OpenAI( base_url="https://gateway.letsur.ai/v1", api_key="YOUR_API_KEY" )
response = client.chat.completions.create( model="gemini-2.5-flash-image", messages=[ {"role": "user", "content": "푸른 하늘 아래 달리는 자동차의 이미지를 만들어줘"} ] )
print(response)
```

- Input = Text + image_config
 - ▶ image_config 파라미터 (선택)

```
response = client.chat.completions.create( model="gemini-2.5-flash-image", messages=[ {"role": "user", "content": "푸른 하늘 아래 달리는 자동차의 이미지를 만들어줘"} ], extra_body={ "image_config": { "aspect_ratio": "16:9", "image_size": "4K" } } )
```

- Input = Text + Image

```
import base64
from openai import OpenAI
import os
from pathlib import Path
import mimetypes

client = OpenAI( base_url="https://gateway.letsur.ai/v1/", api_key="YOUR_API_KEY" )
query = "내 자동차가 하늘을 달리도록 이미지를 편집해줘"
# 이미지 파일을 data URL로 인코딩하여 content 배열에 포함
image_path = Path("/Users/xxx.jpg")
if not image_path.exists():
    raise FileNotFoundError(f"이미지 파일을 찾을 수 없습니다: {image_path}")
with open(image_path, "rb") as f:
    b64 = base64.b64encode(f.read()).decode("utf-8")
mime, _ = mimetypes.guess_type(str(image_path))
mime = mime or "image/png"
messages = [ {"role": "user", "content": [ {"type": "text", "text": query}, {"type": "image_url", "image_url": {"url": f"data:{mime};base64,{b64}"}} ], } ]
resp = client.chat.completions.create(messages=messages, model="gemini-2.5-flash-image")
print(response)
```

✅ Response 예시

응답에 이미지가 포함되어 있을 경우, 기존 구조에 `images` 필드가 추가됩니다:


```
{ "id": "chatcmpl-123", "object": "chat.completion", "created": 1739000000, "choices": [ { "message": { "role": "assistant", "content": "다음은 생성된 이미지입니다." } } ], "images": [ { "image_url": { "url": "iVBORw0KGgoAAAANSUhEUgAA...A..." } }, ... ] }
```



images 필드는 base64로 인코딩된 이미지 배열이며, 클라이언트에서 직접 디코딩하여 파일로 저장할 수 있습니다.

6.3. Python 예제 코드

```
import base64 from io import BytesIO from PIL import Image from openai import OpenAI import os client = OpenAI( base_url="https://gateway.letsur.ai/v1", api_key="YOUR_API_KEY" ) query = "푸른 하늘 아래 달리는 자동차의 이미지를 만들어줘" query_message = [{"role": "user", "content": query}] resp = client.chat.completions.create(messages=query_message, model="gemini-2.5-flash-image") image_with_web_base64 = resp.choices[-1].message.images[0]["image_url"]["url"] _, base64_encoded_string = image_with_web_base64.split(",", 1) decoded_image_bytes = base64.b64decode(base64_encoded_string) image_stream = BytesIO(decoded_image_bytes) image = Image.open(image_stream) image.show()
```

6.4 TypeScript 예제 코드

```
import OpenAI from "openai"; const client = new OpenAI({ baseUrl: "https://gateway.letsur.ai/v1", apiKey: "YOUR_API_KEY", }); const response = await client.chat.completions.create({ model: "gemini-3-pro-image-preview", messages: [{ role: "user", content: "A cute cat" }], image_config: { aspect_ratio: "16:9", image_size: "4K" }, } as any); console.log(response);
```



TypeScript OpenAI SDK에서는 `extra_body` 를 사용하지 않습니다.

`image_config` 를 최상위 파라미터로 직접 전달하고, `as any` 로 타입 캐스팅합니다.



잘못된 예시

```
{ extra_body: { image_config: { ... } } }
```



올바른 예시

```
{ image_config: { aspect_ratio: "16:9", image_size: "4K" } } as any
```

✅ 요약

- `/chat/completions` 그대로 사용 → 완전 호환
- 초저지연 text/image 생성
- base64 이미지 응답 (`images[]`)
- `model="gemini-2.5-flash-image"` `model="gemini-3-pro-image-preview"` 로 지정